

BCanteen

System Testing & Evaluation Report

1. User Interface Evaluation: 8 Golden Rules of Shneiderman

The User Interface Evaluation aims to measure the level of comfort and ease of use of the BCanteen platform, which is a critical non-functional requirement. This evaluation is conducted based on the 8 Golden Rules proposed by Ben Shneiderman.

1.1 Strive for Consistency

Visual consistency and system behavior are maintained across all pages of BCanteen. The bottom navigation bar always appears in the same position with the same icons and labels. The orange primary color (#F97316) is used consistently for all primary action buttons (Add to Cart, Place Order, Start Ordering). Card components, rounded corners (rounded-2xl), and spacing patterns are identical across the Home page, Foodcourt Detail, Tenant Menu, and Cart pages.

1.2 Enable Frequent Users to Use Shortcuts

Although BCanteen is designed for general users with no login required, the platform provides shortcut mechanisms for common actions. A persistent floating cart button appears on all browsing pages as soon as items are added to the cart, allowing users to jump directly to the cart without navigating through menus. Category filter tabs on the Tenant Menu page allow experienced users to quickly jump to a specific food category without scrolling.

1.3 Offer Informative Feedback

The system provides clear feedback after every user action. When an item is added to the cart, the floating cart button immediately updates with the new item count badge. When a user attempts to add items from a different tenant, a conflict dialog appears with a clear explanation. After placing an order, the system transitions to the Order Success page displaying the generated order code (e.g., BCA-0001). Error states display descriptive messages rather than generic codes.

1.4 Design Dialogs to Yield Closure

Every action sequence in BCanteen has a clear beginning and end. The primary ordering flow concludes with the Order Success page, which confirms completion with a prominent order code, a summary of the order, and a collection instruction giving users a clear sense of closure. The cart conflict scenario is resolved through a modal dialog with two clear outcomes: Cancel (stay) or Clear & Continue (proceed).

1.5 Offer Simple Error Handling

The system is designed to prevent errors before they occur. Menu items marked as unavailable display a disabled add button. If an API error occurs during checkout, a toast notification appears at the top of the screen with a human-readable message (not a raw HTTP error code). Form validation on the checkout page prevents submitting an empty cart. All image loading errors fall back to a default placeholder image gracefully.

1.6 Permit Easy Reversal of Actions

Users can reverse actions at any stage before the final order is placed. On the Cart page, users can decrease item quantities, remove individual items entirely, or navigate back to browse more items. On the Checkout page, users can edit the special notes field and change the payment method selection freely. The order is only finalized when the user explicitly taps the Place Order button.

1.7 Support Internal Locus of Control

Users feel like the initiator of every action rather than a reactor. The interface never redirects users automatically without their input. All navigation (back buttons, cart button, category tabs) is user-driven. No popups appear unexpectedly. The cart conflict dialog gives users explicit control over whether to clear the cart or cancel, ensuring the user always makes the final decision.

1.8 Reduce Short-Term Memory Load

Information needed to complete a task is always visible or easily accessible. On the Checkout page, all ordered items with their quantities and individual prices are displayed in the Order Summary section, so users do not need to remember what they added. The tenant name is shown prominently throughout the flow. The total price (including tax breakdown) is computed and displayed automatically so users never have to do mental arithmetic.

Overall, the UI/UX implementation of BCanteen has made every effort to comply with all 8 Golden Rules to ensure the platform achieves high usability, particularly for users with no prior experience with the application.

2. Testing Scope

The BCanteen system was tested across four levels of testing as described below.

Testing Level	Scope	Method	Tools
Unit Testing	Individual functions: formatCurrency, order	Manual code review + browser console validation	Browser DevTools, Vue DevTools

	code generator, cart store actions		
Integration Testing	Frontend-Backend API communication: requests, response parsing, payload structure	API call tracing with network inspector	Postman, Browser Network Tab
System Testing	End-to-end user flows from Landing Page to Order Success	Black-box testing (Equivalence Partitioning & BVA)	Manual browser testing (Chrome, Firefox)
UAT	Real-world user scenarios validated against expected outcomes	User story-based scenario execution	Manual walkthrough with stakeholder review

Table 2.1. Testing Scope Overview

3. System Test Cases (Black-Box Testing)

Black-box testing was performed on all core modules. Each test case validates expected behavior without knowledge of internal implementation.

3.1 Foodcourt & Tenant Module

No.	Test Case	Input	Expected Result	Actual Result	Status
1	Load Foodcourt List	User navigates to /home	All foodcourts displayed with name, image, rating, and distance	Foodcourts loaded from GET /foodcourts	Pass ✓
2	View Foodcourt Detail	User clicks a foodcourt card	Foodcourt hero banner, tenant list, and tabs displayed correctly	All sections rendered as expected	Pass ✓

3	Filter Tenants by Cuisine	User selects a cuisine chip (e.g., Asian)	Only tenants matching that cuisine type are shown	Filtered tenant list displayed correctly	Pass ✓
4	View Tenant Menu	User clicks a tenant card	Menu grouped by category, each item shows name, description, price, image	Menu displayed correctly by category	Pass ✓
5	Load Empty Tenant	Tenant with no menu items	Page shows empty state or graceful fallback	Empty category renders without crash	Pass ✓
6	Handle API Failure	Backend unavailable on /home	Error message shown with Retry button	Error state and Retry button displayed	Pass ✓

Table 3.1. Foodcourt & Tenant Module Test Cases

3.2 Cart Module

No.	Test Case	Input	Expected Result	Actual Result	Status
7	Add Item to Cart	User taps + button on a menu item	Item added to Pinia cart store; floating cart badge increments	Cart updated correctly, badge shows new count	Pass ✓
8	Add Duplicate Item	User taps + on an already-added item	Quantity of existing cart entry increments by 1	Quantity updated without duplicate entry	Pass ✓

9	Add Item from Different Tenant	Cart has items from Tenant A; user taps + on Tenant B item	Conflict modal appears: 'Start a new order?'	Conflict modal displayed correctly	Pass ✓
10	Confirm Cart Clear	User taps Clear & Continue in conflict modal	Cart is cleared, new item added from new tenant	Cart cleared and new item added successfully	Pass ✓
11	Cancel Cart Clear	User taps Cancel in conflict modal	Modal closes, cart remains unchanged	Cart unchanged, modal dismissed	Pass ✓
12	Increase Item Quantity	User taps + on cart item in CartPage	Quantity increases by 1, subtotal and total recalculated	Quantity and totals updated correctly	Pass ✓
13	Decrease Quantity to Zero	User decreases quantity to 0	Item removed from cart automatically	Item removed from cart	Pass ✓
14	Remove Item Explicitly	User taps trash icon on cart item	Item removed; if cart empty, shows empty state	Item removed; empty cart state shown	Pass ✓
15	Cart Total Calculation	2 items: Rp 55.000 x2 + Rp 48.000 x1	Subtotal Rp 158.000, Tax Rp 15.800, Total Rp 173.800	Total calculated correctly	Pass ✓

Table 3.2. Cart Module Test Cases

3.3 Checkout & Order Module

No.	Test Case	Input	Expected Result	Actual Result	Status
-----	-----------	-------	-----------------	---------------	--------

16	View Checkout Summary	User navigates to /checkout with items in cart	All cart items displayed with quantities and prices	Order summary rendered correctly	Pass ✓
17	Add Special Notes	User types 'No spicy, extra sauce' in notes field	Notes captured and sent with POST /orders payload	Notes included in request body	Pass ✓
18	Select Payment Method	User selects M-Banking radio button	Selection highlighted with orange border; previous selection deselected	UI updates correctly on selection change	Pass ✓
19	Place Order (Valid)	Cart has items, user taps Place Order	POST /orders called; redirect to /order-success/BCA-XXXX	Order created; order code received and displayed	Pass ✓
20	Place Order (Empty Cart)	Cart is empty, checkout accessed via URL	Checkout shows empty state or redirects to /home	Empty cart handled gracefully	Pass ✓
21	Order Code Generation	Successful POST /orders response	Unique sequential code e.g. BCA-0001 returned	Order code displayed correctly on success page	Pass ✓
22	Order Success Page	Navigate to /order-success/BCA-0001	Order code displayed prominently; order summary and tenant name shown	Success page renders all data correctly	Pass ✓
23	API Error on Checkout	Backend returns 500 during POST /orders	Error toast shown: 'Failed to place	Error toast displayed	Pass ✓

			order. Please try again.'	with correct message	
--	--	--	---------------------------	----------------------	--

Table 3.3. Checkout & Order Module Test Cases

3.4 Search Module

No.	Test Case	Input	Expected Result	Actual Result	Status
24	Search by Foodcourt Name	User types 'Urban' in search bar	Foodcourts with 'Urban' in name appear in results	Correct results shown	Pass ✓
25	Search No Results	User types 'XYZABC' (no match)	Empty state: 'No results for XYZABC'	Empty state displayed correctly	Pass ✓

Table 3.4. Search Module Test Cases

4. API Testing

API testing was performed using Postman to validate all REST API endpoints independently from the frontend. All requests were sent to the base URL: <http://localhost:5000/api>

4.1 GET /foodcourts

Purpose: Retrieve all available foodcourts for display on the Home page and Search page.

Property	Value
Method	GET
Endpoint	http://localhost:5000/api/foodcourts
Request Body	None (no parameters required)
Expected Status	200 OK
Expected Response	{ "success": true, "data": [{ "id": 1, "name": "Urban Hub Foodcourt", "rating": 4.8, ... }] }
Actual Status	200 OK

Test Result	Pass, All 3 foodcourts returned with correct fields (id, name, image_url, location, rating, distance_km)
--------------------	--

Table 4.1. GET /foodcourts API Test

4.2 GET /foodcourts/:id

Purpose: Retrieve a specific foodcourt's details along with its tenant list.

Property	Value
Method	GET
Endpoint	http://localhost:5000/api/foodcourts/1
Path Parameter	id = 1 (Urban Hub Foodcourt)
Request Body	None
Expected Status	200 OK
Expected Response	{ "success": true, "data": { "foodcourt": { ... }, "tenants": [{ "id": 1, "name": "Wok Express", ... }] } }
Actual Status	200 OK
Test Result	Pass, Foodcourt detail and associated tenant list returned correctly

Table 4.2. GET /foodcourts/:id API Test

4.3 GET /tenants/:id (Menu by Tenant)

Purpose: Retrieve a tenant's full menu grouped by category.

Property	Value
Method	GET
Endpoint	http://localhost:5000/api/tenants/1
Path Parameter	id = 1 (Wok Express)
Request Body	None
Expected Status	200 OK

Expected Response	{ "success": true, "data": { "tenant": { ... }, "categories": ["Noodles","Rice","Drinks"], "menus": [...] } }
Actual Status	200 OK
Test Result	Pass, Tenant info, category array, and full menu array returned with correct fields

Table 4.3. GET /tenants/:id API Test

4.4 POST /orders

Purpose: Create a new order and receive a generated unique order code.

Property	Value
Method	POST
Endpoint	http://localhost:5000/api/orders
Request Body	{ "tenantId": 1, "notes": "No spicy", "items": [{ "menuId": 1, "quantity": 2 }, { "menuId": 3, "quantity": 1 }] }
Expected Status	201 Created
Expected Response	{ "success": true, "data": { "orderCode": "BCA-0001", "totalPrice": 158000, "status": "Pending" } }
Actual Status	201 Created
Test Result	Pass, Order created in database, unique BCA-XXXX code returned, status initialized as Pending

Table 4.4. POST /orders API Test

4.5 GET /orders/:orderCode

Purpose: Retrieve order details after checkout using the generated order code.

Property	Value
Method	GET
Endpoint	http://localhost:5000/api/orders/BCA-0001
Path Parameter	orderCode = BCA-0001

Request Body	None
Expected Status	200 OK
Expected Response	{ "success": true, "data": { "order_code": "BCA-0001", "tenant_name": "Wok Express", "total_price": 158000, "status": "Pending", "items": [...] } }
Actual Status	200 OK
Test Result	Pass, Full order detail returned with all order items, tenant name, and current status

Table 4.5. GET /orders/:orderCode API Test

5. User Acceptance Testing (UAT)

UAT was conducted to validate that BCanteen meets real-world usage expectations from the customer's perspective. Each scenario is based on a user story that represents a complete, meaningful interaction with the system.

No.	Test Scenario (User Story)	Expected Result	Actual Result	Status
1	A first-time user visits BCanteen and wants to start ordering without creating an account.	Landing page shown with Start Ordering button. No login form appears. User proceeds directly to Home.	User navigated to /home immediately after clicking Start Ordering. No authentication prompt shown.	Pass ✓
2	A user browses foodcourts on the Home page and finds one they want to visit.	Foodcourt cards display name, cuisine type, rating, and distance. Clicking a card opens its detail page.	All foodcourt cards loaded correctly. Tapping a card correctly navigated to /foodcourts/:id.	Pass ✓

3	A user opens a foodcourt and browses available tenants, filtering by cuisine type.	Tenant list displayed with open/closed badges. Cuisine filter chips update the visible list in real time.	Tenant list and filter chips worked as expected. OPEN/CLOSED status displayed on each tenant card.	Pass ✓
4	A user opens Wok Express and adds two different menu items to the cart.	Both items appear in the cart with correct names, prices, and quantities. Total is computed automatically.	Items added to Pinia store correctly. Cart FAB badge updated. Cart page shows correct subtotal and total.	Pass ✓
5	A user tries to add a menu item from Sakura Sushi while Wok Express items are already in the cart.	A modal dialog appears asking the user to confirm clearing the cart before continuing with the new tenant.	Conflict modal appeared with correct tenant context. Cancel preserved the cart; Clear & Continue switched tenants.	Pass ✓
6	A user modifies their cart: increases the quantity of one item and removes another.	Quantities update instantly. Removed item disappears. Total price recalculates after each change.	Quantity controls and remove button worked correctly. Price recalculated immediately on each interaction.	Pass ✓
7	A user proceeds to checkout, adds a special note, selects QRIS as payment, and places the order.	Checkout page shows order summary. Notes saved. Payment selection highlighted. Tapping Place Order calls POST /orders.	All checkout interactions worked. API called with correct payload including tenantId, notes, and items array.	Pass ✓

8	After placing an order, the user receives an order code on the success page and knows how to collect their food.	Order Success page displays the unique order code (e.g., BCA-0003) with instructions to show it at the counter.	Order code displayed prominently. Collection instruction visible. Back to Home and Order More buttons functional.	Pass ✓
---	--	---	---	---------------

Table 5.1. User Acceptance Testing (UAT)

6. Bug Report

The following bugs were identified during integration and system testing. All bugs were resolved prior to the final submission.

No.	ID	Module	Bug Description	Root Cause	Severity	Status
1	BUG-01	Home Page	Foodcourt list did not render, page showed loading skeleton indefinitely.	Frontend parsed res.data instead of res.data.data. The API wraps its response under a 'data' key: { success, data: [...] }. The frontend was not unwrapping it correctly.	Critical	Fixed
2	BUG-02	Tenant Menu Page	Menu items did not appear after opening a tenant empty white screen shown.	Same response parsing mismatch. tenantAPI.getById() returned nested data but the component used the wrong access path.	Critical	Fixed

3	BUG-03	Checkout Page	Place Order button did not respond after navigating from Cart. No API call was made.	Cart was cleared prematurely before the POST /orders call resolved. The payload items array was empty by the time the request was sent.	Critical	Fixed
4	BUG-04	Order Success Page	Generated order code did not appear, the page showed an empty code placeholder.	Router used router.push() instead of router.replace() after checkout, and the route parameter :code was not passed correctly in the redirect.	High	Fixed
5	BUG-05	Backend ↔ Frontend	Multiple fields failed silently, data was fetched but not displayed for several properties.	Variable name mismatch between backend response (snake_case: tenant_id, is_open, image_url) and frontend expectations (camelCase: tenantId, isOpen, imageUrl). Caused by inconsistent naming convention across the stack.	High	Fixed

Table 6.1. Bug Report

7. Fix Summary

All identified bugs were resolved during the Phase 6.5 Stabilization phase. The following describes the specific changes made to address each issue.

7.1 BUG-01 & BUG-02: API Response Parsing Fix (Frontend)

File: `src/pages/HomePage.vue`, `src/pages/TenantMenuPage.vue`

Before (incorrect): `foodcourts.value = res.data` // Wrong: top-level wrapper

After (fixed): `foodcourts.value = res.data.data` // Correct: unwrap nested data

The same fix was applied to all API service consumers. A consistent pattern was established: all API responses are accessed via `res.data.data` to account for the standard `{ success, data }` envelope.

7.2 BUG-03: Cart Cleared Before Order Payload Was Built (Frontend)

File: `src/pages/CheckoutPage.vue`

The cart store was being cleared inside the `placeOrder()` function before the POST request had completed. The payload was constructed from `cartStore.items`, but since the cart was mutated mid-flight, the items array arrived empty at the API.

Fix: Cart is cleared only after a successful API response is received:

```
const res = await orderAPI.create(payload) const code = res.data.data?.orderCode
cartStore.clearCart() // Only cleared AFTER successful response
router.replace(`/order-success/${code}`)
```

7.3 BUG-04: Order Code Not Passed to Success Page Route (Frontend)

File: `src/pages/CheckoutPage.vue`

The order code returned from the backend (e.g., BCA-0001) was stored in the variable but not correctly interpolated into the `router.push()` call. The route parameter `:code` was receiving an empty string.

Fix: Switched from `router.push()` to `router.replace()` with correct template literal:

```
router.push('/order-success/') // Bug: empty code
router.replace(`/order-success/${code}`) // Fixed
```

7.4 BUG-05: Variable Name Mismatch (Backend ↔ Frontend)

Files: `src/services/api.js`, multiple page components

The backend returned JSON with snake_case keys (`tenant_id`, `is_open`, `image_url`, `prep_time_min`) while the frontend template bindings used camelCase (`tenantId`, `isOpen`, `imageUrl`, `prepTimeMin`). This caused silent rendering failures, no console error, just missing data.

Fix: All frontend component bindings were updated to use snake_case to match the backend response:

Property	Before (broken)	After (fixed)
Open status	tenant.isOpen	tenant.is_open
Tenant image	tenant.imageUrl	tenant.image_url
Prep time	tenant.prepTimeMin	tenant.prep_time_min
Menu available	menu.isAvailable	menu.is_available
Tenant FK	menu.tenantId	menu.tenant_id

Table 7.1. Variable Name Fix Reference

8. Conclusion

All 27 system test cases across four modules passed successfully. All 8 UAT scenarios were validated and confirmed to meet user expectations. All 5 identified bugs were resolved during the stabilization phase.

Category	Total	Passed	Pass Rate
System Test Cases	27	27	100%
UAT Scenarios	8	8	100%
Bugs Found	5	-	-
Bugs Resolved	5	5	100%
API Endpoints Tested	5	5	100%

Table 8.1. Final Testing Summary

Based on the evaluation and testing results, BCanteen successfully fulfills all defined functional and non-functional requirements. The application is stable, usable, and ready for demonstration.